



# LLM Alignment: Instruction Tuning, RLHF, and DPO



Mohammad Javad Dousti

# Motivation

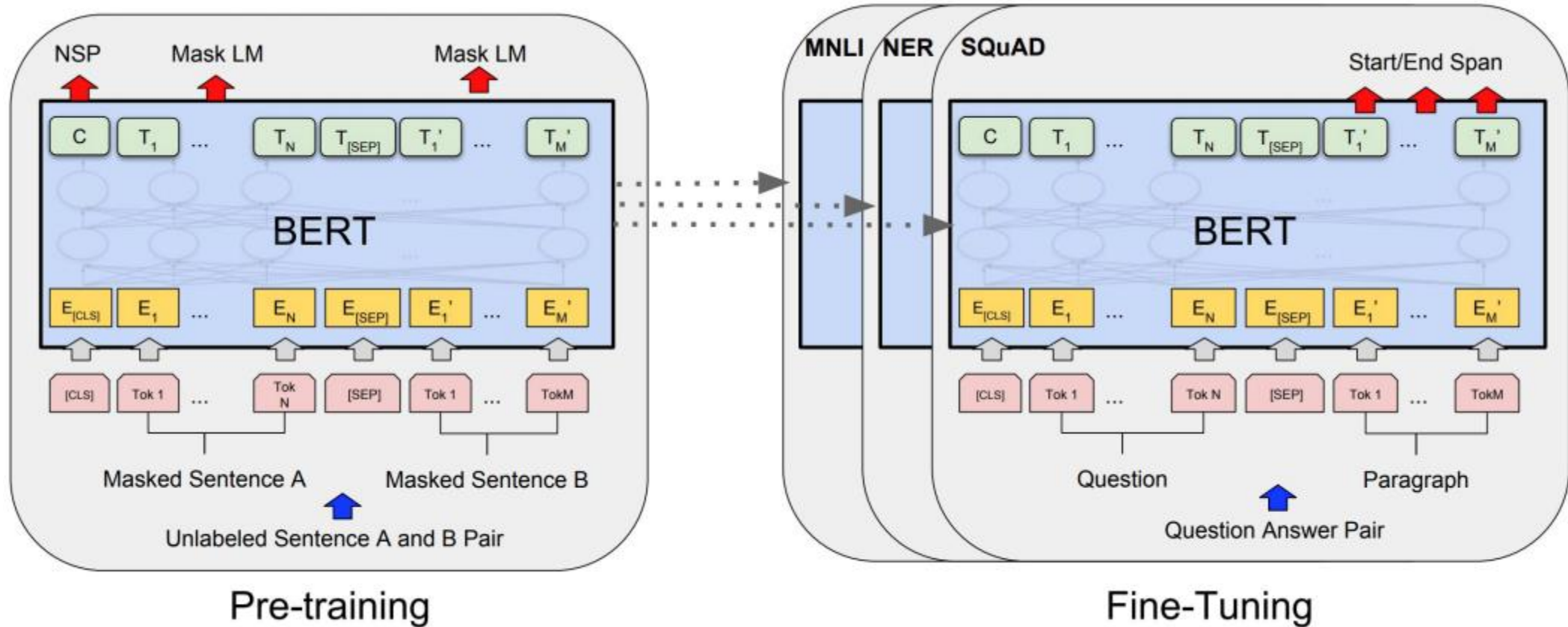
---

□ Let's compare these two models:

- <https://api.together.xyz/playground/language/meta-llama/Llama-2-7b-hf>
- <https://api.together.xyz/playground/chat/meta-llama/Llama-2-7b-chat-hf>



# BERT, 2018



# Limitations of Pretraining → Fine-tuning

---

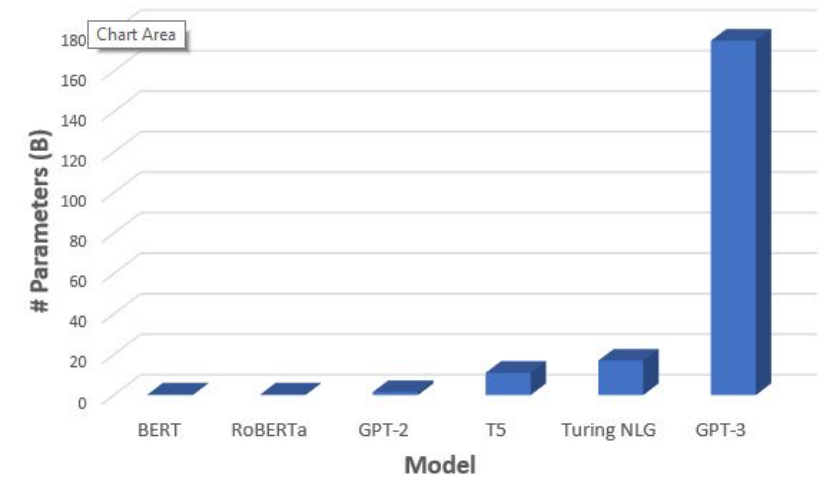
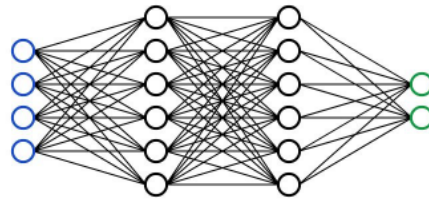
- ❑ Practical issues
  - Needs large task-specific datasets
  - A copy of model for each task
- ❑ Potential to exploit spurious correlations
  - Models can overfit to training distributions and don't generalize to out of distribution datasets
  - Learning datasets and not the underlying task
- ❑ Humans don't need large supervised datasets
  - We learn from few examples and/or an instruction
  - We switch between tasks easily

# self-supervised learning

+

# scale

In 1885, Stanford University was \_\_\_\_\_



# Scale in #parameters



Scaling doesn't solve all problems.  
We also need post-training

# We can't talk to the pretrained model directly

---

## Model input

Make up a word that means "when two AI researchers go on a date".

## PaLM 540B output

Make up a word that means "when two AI researchers go on a date".

The day after he was hired, the new programmer wrote an e-mail to all of his fellow programmers. It said, "I will be on vacation next week."

The day after he was hired, the new programmer wrote an e-mail to all of his fellow programmers. It said, "I will be on vacation next week."

The day after [...]

✗ (repeats input and keep repeating generations)

## Flan-PaLM 540B output

date-mining ✓

# Hack: Frame the question so that the answer is the next token

---

Q: The square root of  $x$  is the cube root of  $y$ . What is  $y$  to the power of 2, if  $x = 4$ ?

A:



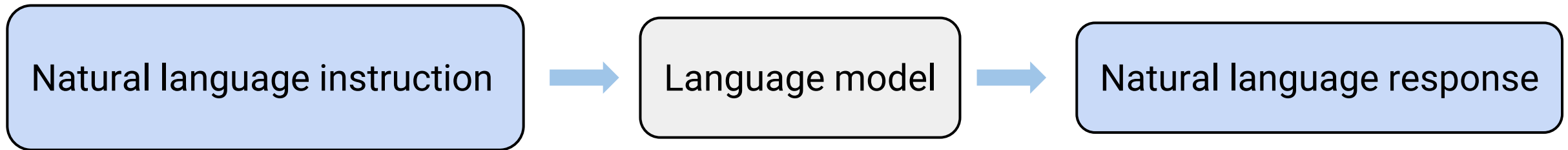
Pretrained model just predicts the next token, which happens to be the answer

# Instruction finetuning

---

Frame **all** tasks in the form of

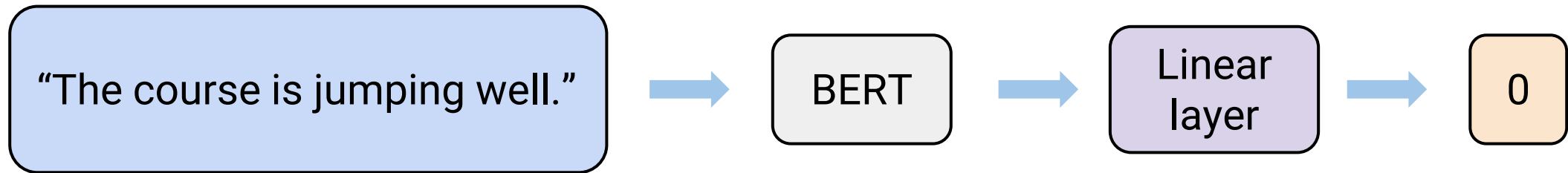
natural language instruction to natural language response mapping



---

**Input: text**

**Output: label**

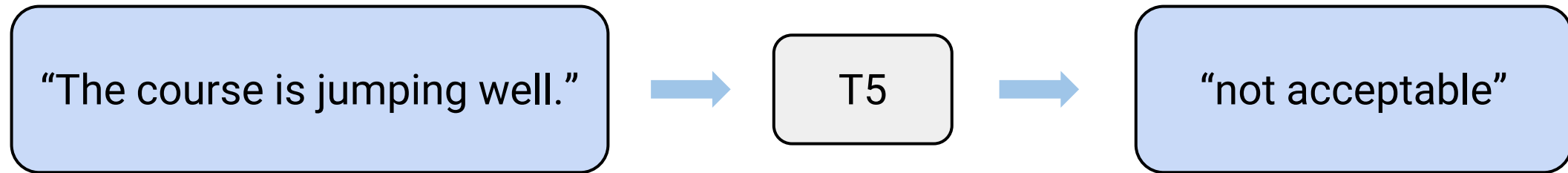


Task specific linear layer is necessary

---

**Input: text**

**Output: text**

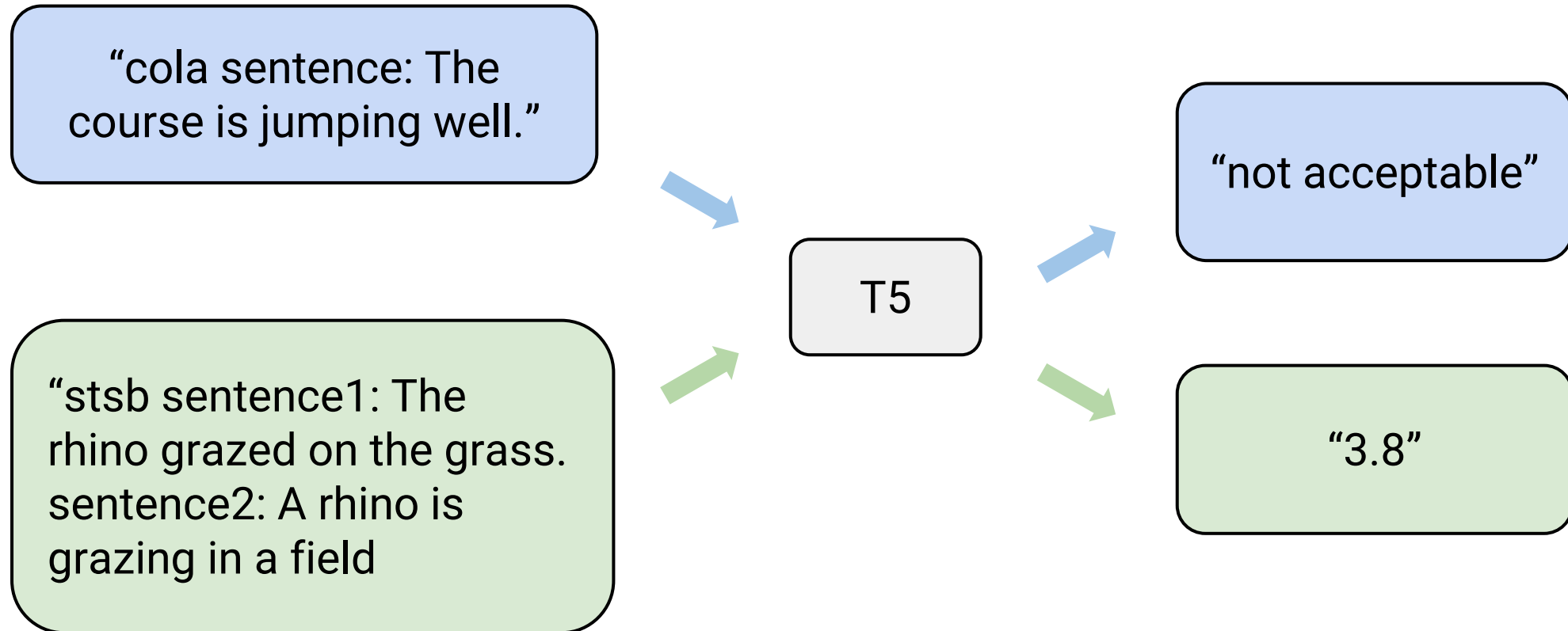


Architecture is unified across tasks with *text-to-text* format

---

**Input: text**

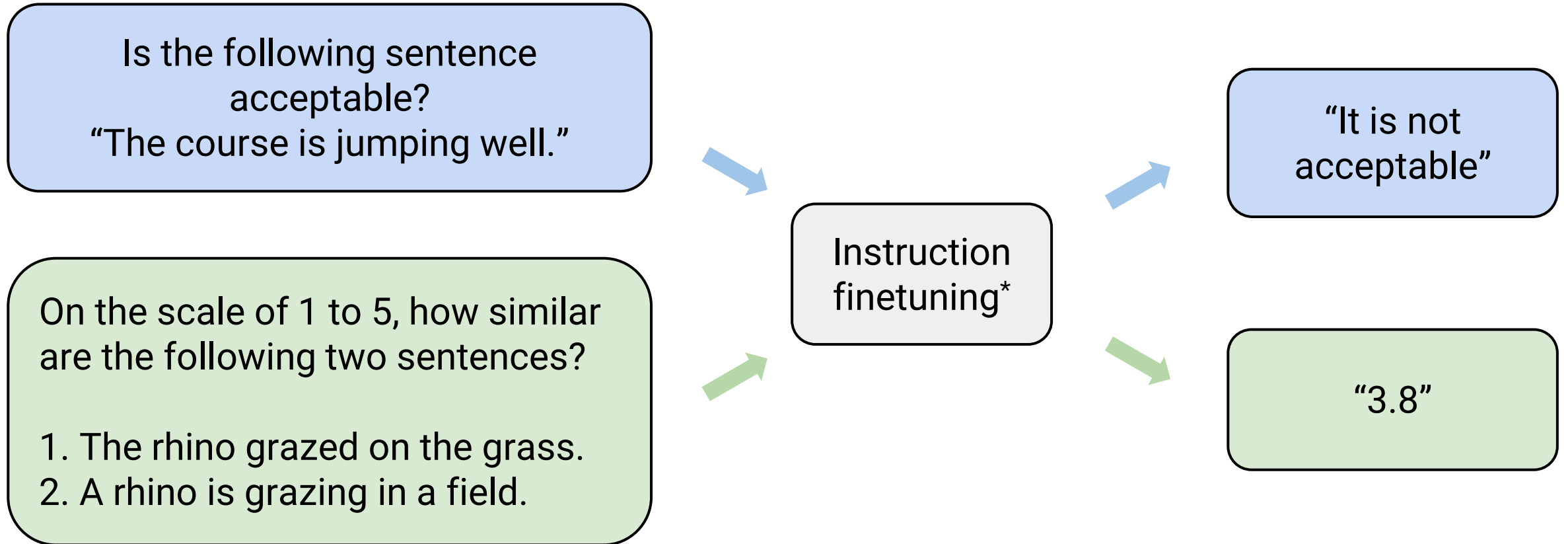
**Output: text**



Tasks are not semantically related

## Input: text

## Output: text



Tasks are unified. So for an unseen task, the model just needs to respond to the natural language instruction

*\*Wei et al. (2021), Sanh et al. (2021), Ouyang et al. (2022)*

## Finetuning tasks

### TO-SF

Commonsense reasoning  
Question generation  
Closed-book QA  
Adversarial QA  
Extractive QA  
Title/context generation  
Topic classification  
Struct-to-text  
...

*55 Datasets, 14 Categories,  
193 Tasks*

### Muffin

|                            |                   |
|----------------------------|-------------------|
| Natural language inference | Closed-book QA    |
| Code instruction gen.      | Conversational QA |
| Program synthesis          | Code repair       |
| Dialog context generation  | ...               |

*69 Datasets, 27 Categories, 80 Tasks*

### CoT (Reasoning)

|                       |                        |
|-----------------------|------------------------|
| Arithmetic reasoning  | Explanation generation |
| Commonsense Reasoning | Sentence composition   |
| Implicit reasoning    | ...                    |

*9 Datasets, 1 Category, 9 Tasks*

### Natural Instructions v2

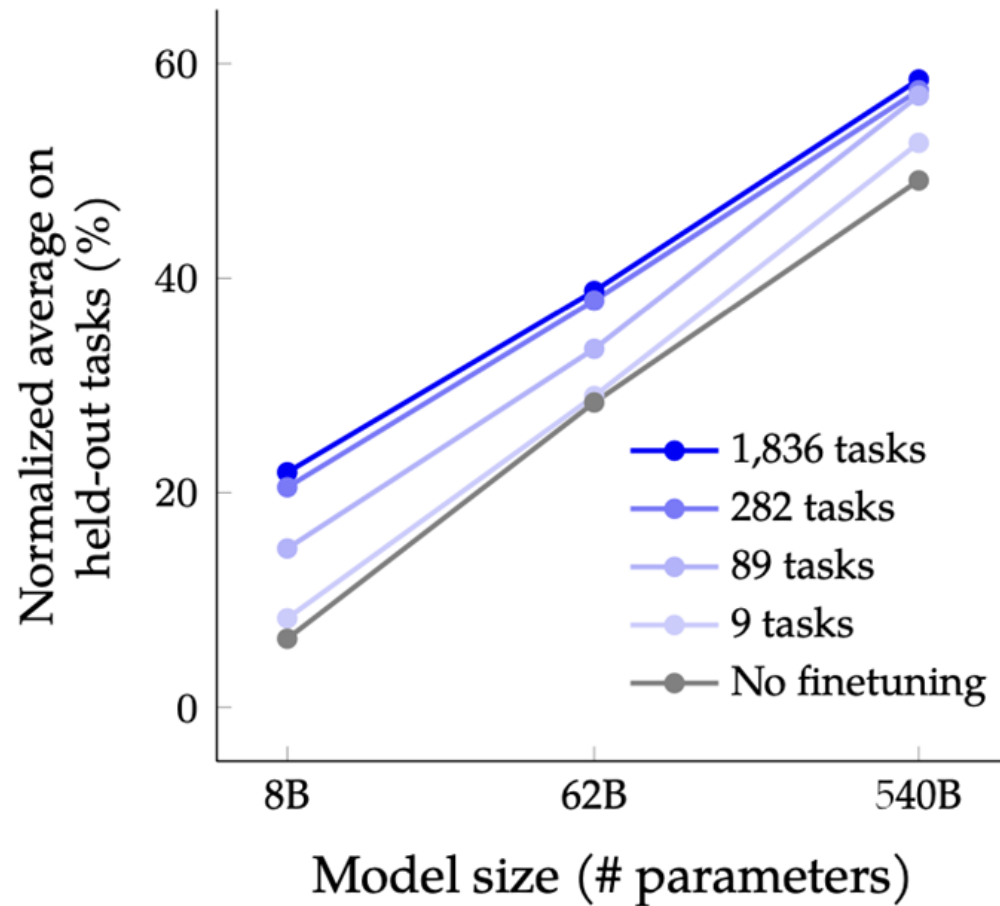
Cause effect classification  
Commonsense reasoning  
Named entity recognition  
Toxic language detection  
Question answering  
Question generation  
Program execution  
Text categorization  
...

*372 Datasets, 108 Categories,  
1554 Tasks*

- ❖ A **Dataset** is an original data source (e.g. SQuAD).
- ❖ A **Task Category** is unique task setup (e.g. the SQuAD dataset is configurable for multiple task categories such as extractive question answering, query generation, and context generation).
- ❖ A **Task** is a unique <dataset, task category> pair, with any number of templates which preserve the task category (e.g. query generation on the SQuAD dataset.)

Instruction finetuning on 1836 (!! ) academic tasks

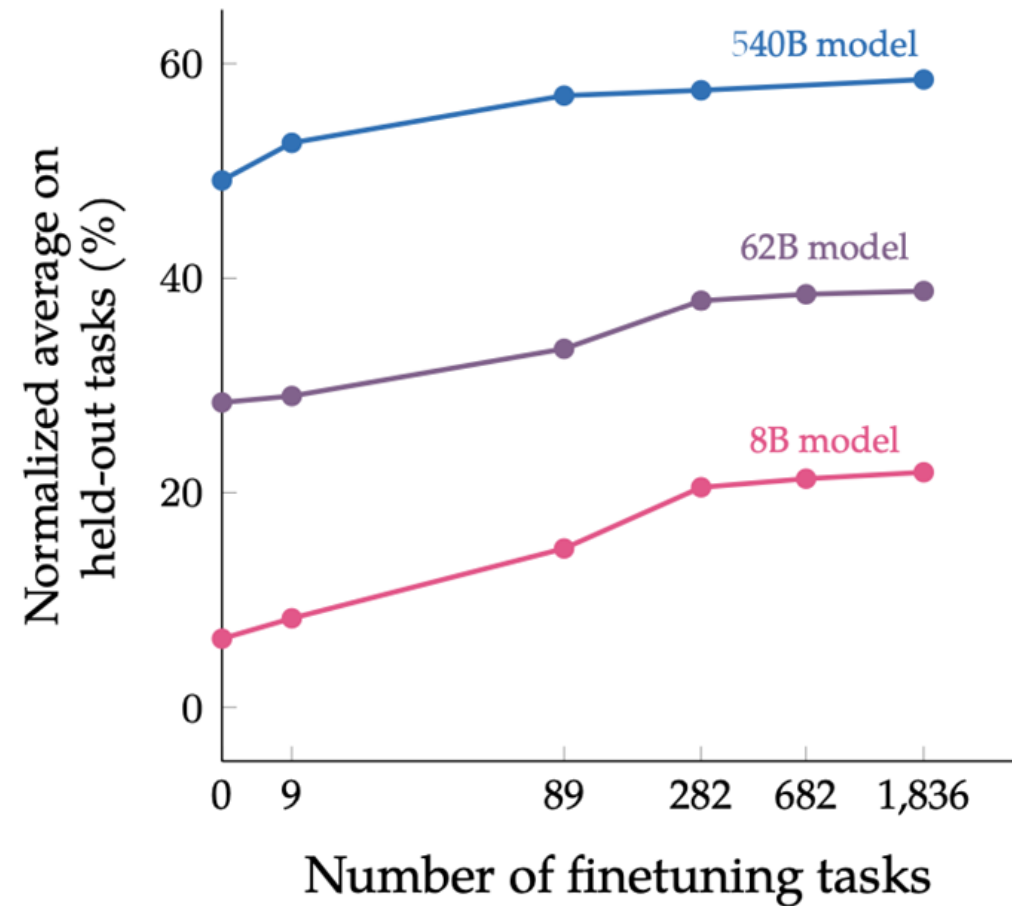
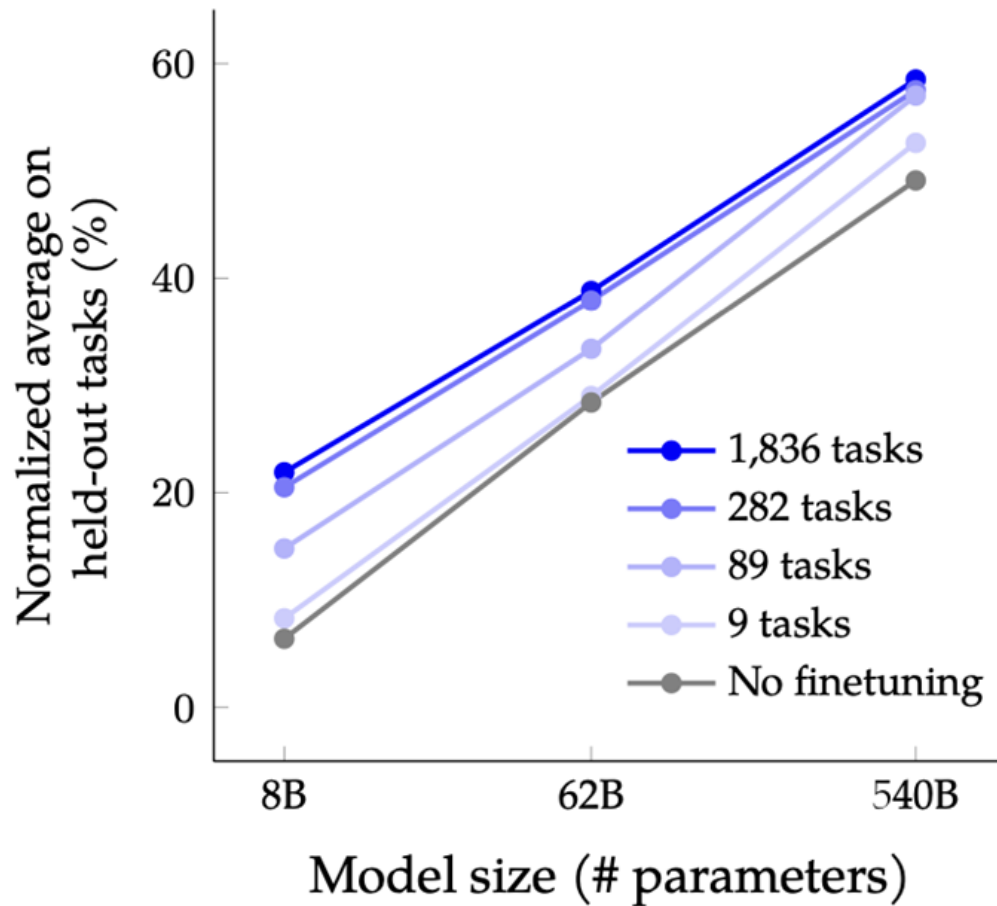
# Scaling the number of tasks and model size improves the performance



*Scaling Instruction-Finetuned Language Models*

*Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus et al. (2022)*<sub>16</sub>

# Scaling the number of tasks and model size improves the performance



*Scaling Instruction-Finetuned Language Models*

*Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus et al. (2022)*<sub>17</sub>

# Instruction finetuning

---

- ❑ Collect examples of (instruction, output) pairs across many tasks and finetune an LM

[FLAN-T5; Chung et al., 2022]

# Methods

---

- ❑ Natural instructions
  - <https://aclanthology.org/2022.acl-long.244/>
- ❑ FLAN
  - <https://arxiv.org/pdf/2109.01652.pdf>
- ❑ Unnatural instructions
  - <https://arxiv.org/pdf/2212.09689.pdf>
- ❑ Self-instruct
  - <https://arxiv.org/abs/2203.02155>
- ❑ InstructGPT

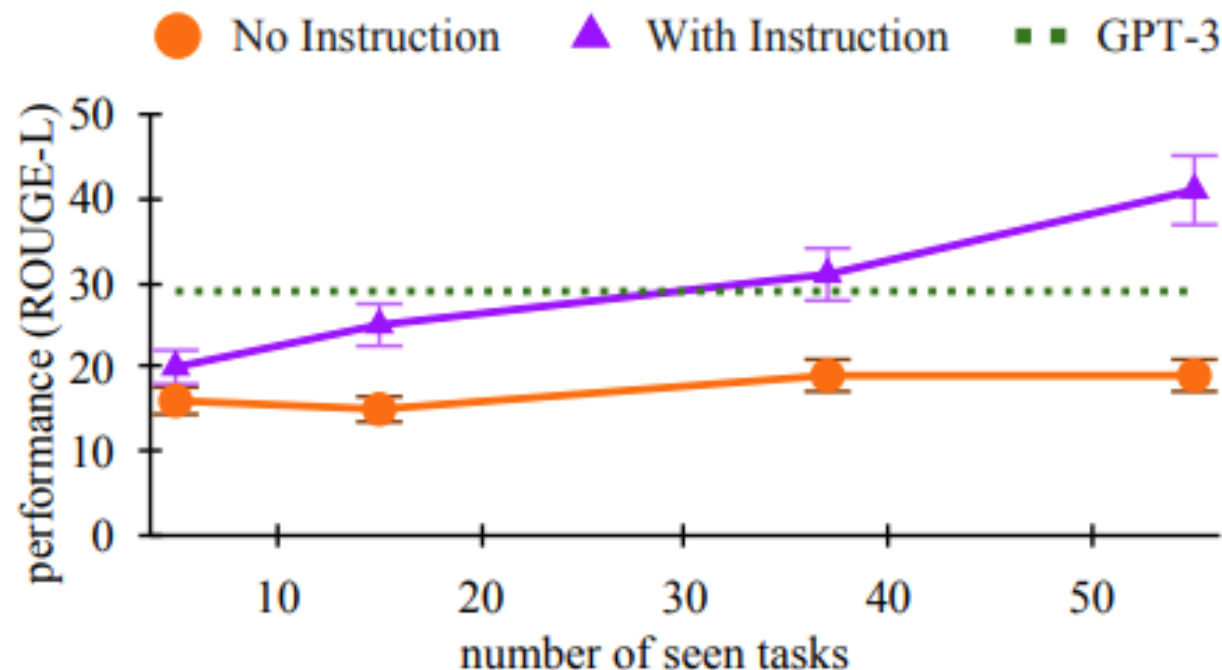
# **Cross-Task Generalization via Natural Language Crowdsourcing Instructions**

**Swaroop Mishra<sup>3\*</sup> Daniel Khashabi<sup>1</sup> Chitta Baral<sup>3</sup> Hannaneh Hajishirzi<sup>1,2</sup>**

<sup>1</sup>Allen Institute for AI   <sup>2</sup>University of Washington   <sup>3</sup>Arizona State University

Natural instruction paper: <https://arxiv.org/pdf/2104.08773.pdf>

# Cross-task generalization



(b) BART evaluation on *unseen* tasks ( $y$ -axis is perf. on  $\mathcal{T}_{\text{unseen}}$ ) when supervised with *seen* tasks ( $x$ -axis is  $|\mathcal{T}_{\text{seen}}|$ ). A model using **instructions** ( $I_t$ ) consistently improves with more observed tasks. In contrast, models with **no access to the instructions** show no sign of improved generalization. Details in §6.3.

# Natural instruction dataset

The format is  
carefully built for  
the instructions

## Instructions for MC-TACO question generation task

- **Title:** Writing questions that involve commonsense understanding of "event duration".
- **Definition:** In this task, we ask you to write a question that involves "event duration", based on a given sentence. Here, event duration is defined as the understanding of how long events typically last. For example, "brushing teeth", usually takes few minutes.
- **Emphasis & Caution:** The written questions are not required to have a single correct answer.
- **Things to avoid:** Don't create questions which have explicit mentions of answers in text. Instead, it has to be implied from what is given. In other words, we want you to use "instinct" or "common sense".

### Positive Example

- **Input:** Sentence: Jack played basketball after school, after which he was very tired.
- **Output:** How long did Jack play basketball?
- **Reason:** the question asks about the duration of an event; therefore it's a temporal event duration question.

### Negative Example

- **Input:** Sentence: He spent two hours on his homework.
- **Output:** How long did he do his homework?
- **Reason:** We DO NOT want this question as the answer is directly mentioned in the text.
- **Suggestion:** -

- **Prompt:** Ask a question on "event duration" based on the provided sentence.

## Example task instances

### Instance

- **Input:** Sentence: It's hail crackled across the comm, and Tara spun to retake her seat at the helm.
- **Expected Output:** How long was the storm?

⋮

### Instance

- **Input:** Sentence: During breakfast one morning, he seemed lost in thought and ignored his food.
- **Expected Output:** How long was he lost in thoughts?

## Instructions

Title

Definition

Things to avoid

Emphasis/caution

Prompt

### Positive Example

Input

Output

Reason

# of **positive examples**

### Negative Example

Input

Output

Reason

Suggestion

# of **negative examples**

## Instances

### Task Instance

Input

Output

# of **instances**

# Self-instruct

---

- ❑ Motivation:
  - Academic tasks do not align with human API usage
  - E.g., responses are usually short (why?)
  - We need to increase the diversity of instructions beyond academic tasks
  - OpenAI did this in InstructGPT.
  - How can we do it with very little access human prompts and responses?
- ❑ Idea: Use LLMs themselves to generate the instructions / answers

# Limitations of instruction-tuning

---

- ❑ One limitation of instruction finetuning is obvious: it's expensive to collect ground truth data for tasks.
- ❑ But there are other, subtler limitations too. Can you think of any?
  - Tasks like open-ended creative generation have no right answer.
    - Write me a story about a dog and her pet grasshopper.
  - Language modeling penalizes all token-level mistakes equally, but some errors are worse than others.
  - Don't learn from negative feedback.
  - Hard to encourage abstaining when the model doesn't know something (avoid hallucination.)
  - Does not directly involve human preferences.
- ❑ Even with instruction finetuning, there a mismatch between the LM objective and the objective of *satisfy human preferences*!
- ❑ Can we explicitly attempt to satisfy human preferences?

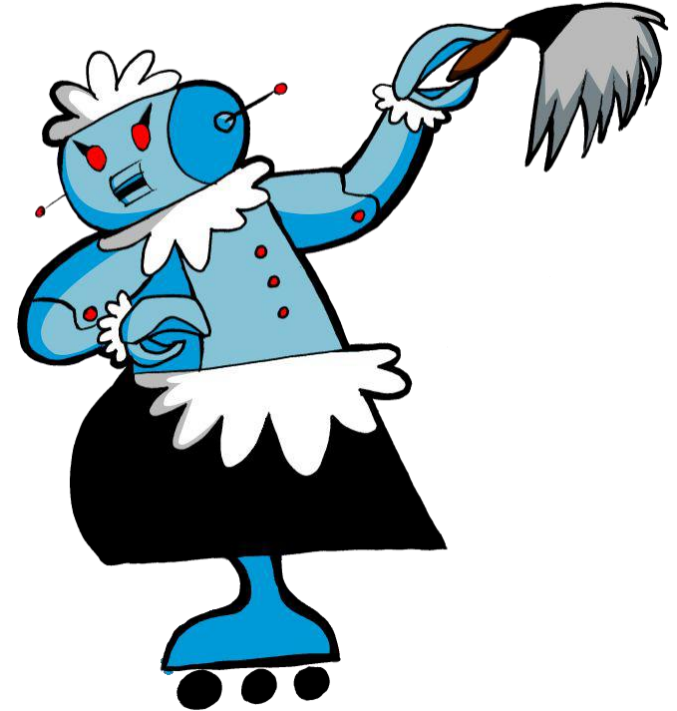
# Motivation: Alignment

# The three H's of Model Desiderata

---

## □ Helpful:

- The AI should help the user solve their task (e.g. answer their questions)



# The three H's of Model Desiderata

---

## ❑ Helpful:

- The AI should help the user solve their task (e.g. answer their questions)

## ❑ Honest:

- The AI should give accurate information
- The AI should express uncertainty when the model doesn't know the answer, instead of hallucinating a wrong answer



# The three H's of Model Desiderata

---

## ❑ Helpful:

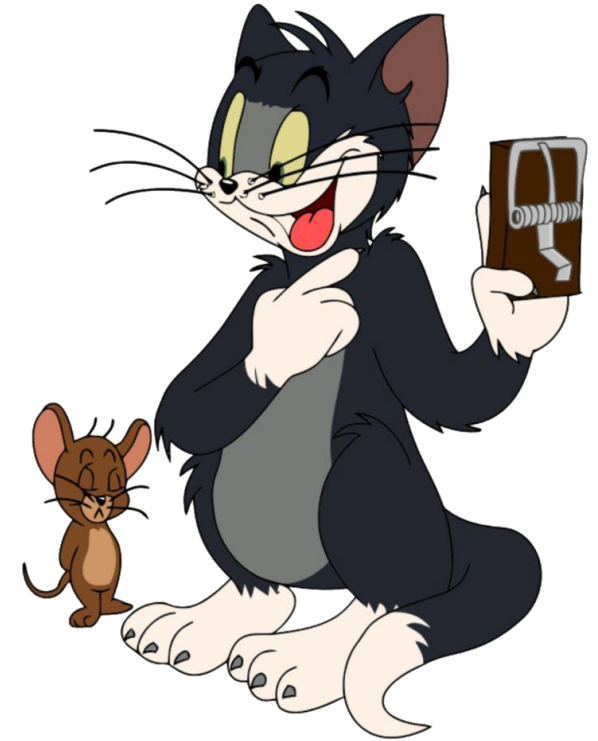
- The AI should help the user solve their task (e.g. answer their questions)

## ❑ Honest:

- The AI should give accurate information
- The AI should express uncertainty when the model doesn't know the answer, instead of hallucinating a wrong answer

## ❑ Harmless:

- The AI should not cause physical, psychological, or social harm to people or the environment

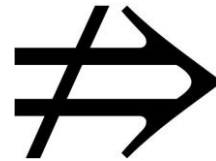


# The Misalignment of Models

- ❑ Misalignment: When the training objective does not capture the desiderata we want from models

$$p(x) = \prod_{i=1}^n p(s_n | s_1, \dots, s_{n-1})$$

**Training:** Predict the next token



The three H's of Model Desiderata

# Optimizing for human preferences

- Let's say we were training a language model on some task (e.g., summarization).
- For each LM sample  $s$ , imagine we had a way to obtain a human reward of that summary:  $R(s) \in \mathbb{R}$ , higher is better.

SAN FRANCISCO,  
California (CNN) --  
A magnitude 4.2  
earthquake shook the  
San Francisco  
...  
overturn unstable  
objects.

An earthquake hit  
San Francisco.  
There was minor  
property damage,  
but no injuries.

$s_1$   
 $R(s_1)=8.0$

The Bay Area has  
good weather but is  
prone to  
earthquakes and  
wildfires.

$s_2$   
 $R(s_2)=1.2$

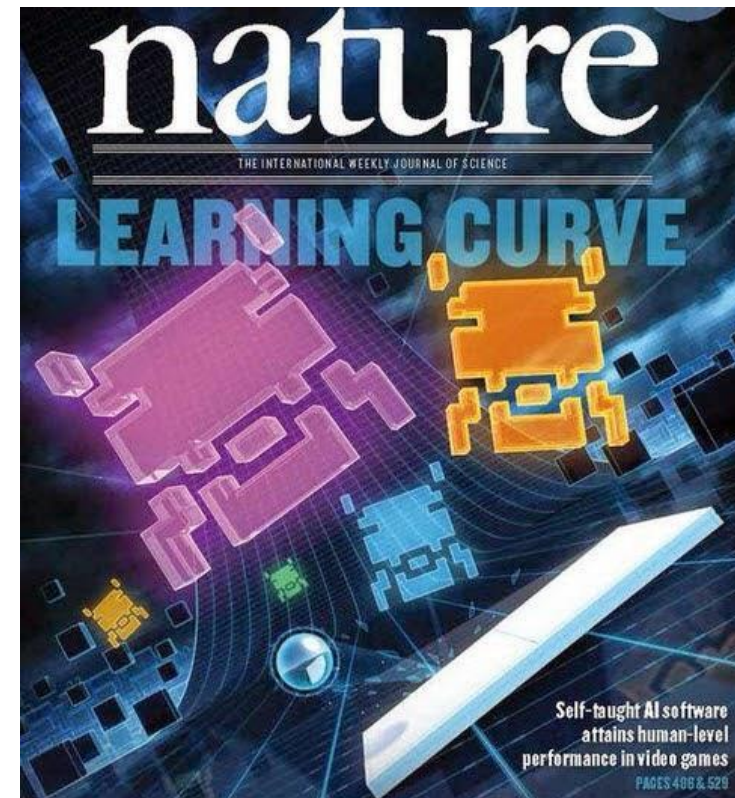
- Now we want to maximize the expected reward of samples from our LM

$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)}[R(\hat{s})]$$

Note: For mathematical simplicity  
we're assuming only one "prompt"

# Reinforcement learning to the rescue

- ❑ The field of **reinforcement learning (RL)** has studied these (and related) problems for many years now [Williams, 1992; Sutton and Barto, 1998]
- ❑ Circa 2013: Resurgence of interest in RL applied to deep learning, game-playing [Mnih et al., 2013]
- ❑ But the interest in applying RL to modern LMs is an even newer phenomenon [Ziegler et al., 2019; Stiennon et al., 2020; Ouyang et al., 2022]. **Why?**
  - RL w/ LMs has commonly been viewed as very hard to get right (still is!)
  - Newer advances in RL algorithms that work for large neural models, including language models (e.g. PPO; [Schulman et al., 2017])



# Optimizing for human preferences

- How do we actually change our LM parameters  $\theta$  to maximize this?

$$\mathbb{E}_{\hat{s} \sim p_{\theta}(s)}[R(\hat{s})]$$

- Let's try doing gradient ascent!

$$\theta_{t+1} := \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}_{\hat{s} \sim p_{\theta_t}(s)}[R(\hat{s})]$$

How do we estimate  
this expectation??



What if our reward  
function is non-  
differentiable??



- **Policy gradient** methods in RL (e.g., REINFORCE; [Williams, 1992]) give us tools for estimating and optimizing this objective.
- For any arbitrary, non-differentiable reward function  $R(s)$ , we can train our language model to maximize expected reward.
- But how to calculate  $R(s)$ ? Human-in-the-loop?

# A (very!) brief introduction to policy gradient/REINFORCE

- We want to obtain

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \nabla_{\theta} \sum_s R(s) p_{\theta}(s) = \sum_s R(s) \nabla_{\theta} p_{\theta}(s)$$

(defn. of expectation) (linearity of gradient)

- Here we'll use a very handy trick known as the **log-derivative trick**. Let's try taking the gradient of  $\log p_{\theta}(s)$

$$\nabla_{\theta} \log p_{\theta}(s) = \frac{1}{p_{\theta}(s)} \nabla_{\theta} p_{\theta}(s) \quad \Rightarrow \quad \nabla_{\theta} p_{\theta}(s) = \nabla_{\theta} \log(p_{\theta}(s)) p_{\theta}(s)$$

(chain rule)

- Plug back in:

$$\begin{aligned} \sum_s R(s) \nabla_{\theta} p_{\theta}(s) &= \sum_s p_{\theta}(s) R(s) \nabla_{\theta} \log p_{\theta}(s) \\ &= \nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})] \end{aligned}$$

This is an expectation of this

# A (very!) brief introduction to policy gradient/REINFORCE

- Now we have put the gradient “*inside*” the expectation, we can approximate this objective with Monte Carlo samples:

$$\nabla_{\theta} \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s})] = \mathbb{E}_{\hat{s} \sim p_{\theta}(s)} [R(\hat{s}) \nabla_{\theta} \log p_{\theta}(\hat{s})] \approx \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta} \log p_{\theta}(s_i)$$

This is why it’s called “**reinforcement learning**”: we **reinforce** good actions, increasing the chance they happen again.

➤ Giving us the update rule:  $\theta_{t+1} := \theta_t + \alpha \frac{1}{m} \sum_{i=1}^m R(s_i) \nabla_{\theta} \log p_{\theta_t}(s_i)$

This is **heavily simplified**! There is a *lot* more needed to do RL w/ LMs. **Can you see any problems with this objective?**

If  $R$  is +++

Take gradient steps to maximize  $p_{\theta}(s_i)$

If  $R$  is ---

Take steps to minimize  $p_{\theta}(s_i)$

# How do we model human preferences?

- ❑ Awesome: now for any **arbitrary, non-differentiable reward function**  $R(s)$ , we can train our language model to maximize expected reward.
- ❑ Not so fast! (Why not?)
- ❑ **Problem 1**: Human-in-the-loop is expensive!
  - **Solution**: Instead of directly asking humans for preferences, model their preferences as a separate (NLP) problem!

An earthquake hit  
San Francisco.  
There was minor  
property damage,  
but no injuries.

$$s_1$$
$$R(s_1) = 8.0$$


The Bay Area has  
good weather but is  
prone to  
earthquakes and  
wildfires.

$$s_2$$
$$R(s_2) = 1.2$$


Train an LM  $RM_\phi(s)$  to  
predict human  
preferences from an  
annotated dataset, then  
optimize for  $RM_\phi$  instead.

# How do we model human preferences?

---

❑ **Problem 2:** Human judgments are noisy and miscalibrated!

- **Solution:** Instead of asking for direct ratings, ask for **pairwise comparisons**, which can be more reliable [Phelps et al., 2015; Clark et al., 2018]

A 4.2 magnitude  
earthquake hit  
San Francisco,  
resulting in  
massive damage.

$$R(s_3) = \begin{matrix} & s_3 \\ 4.1? & 6.6? & 3.2? \end{matrix}$$

# How do we model human preferences?

❑ **Problem 2:** Human judgments are noisy and miscalibrated!

➤ **Solution:** Instead of asking for direct ratings, ask for **pairwise comparisons**, which can be more reliable [Phelps et al., 2015; Clark et al., 2018]

An earthquake hit San Francisco.  
There was minor property damage, but no injuries.

>

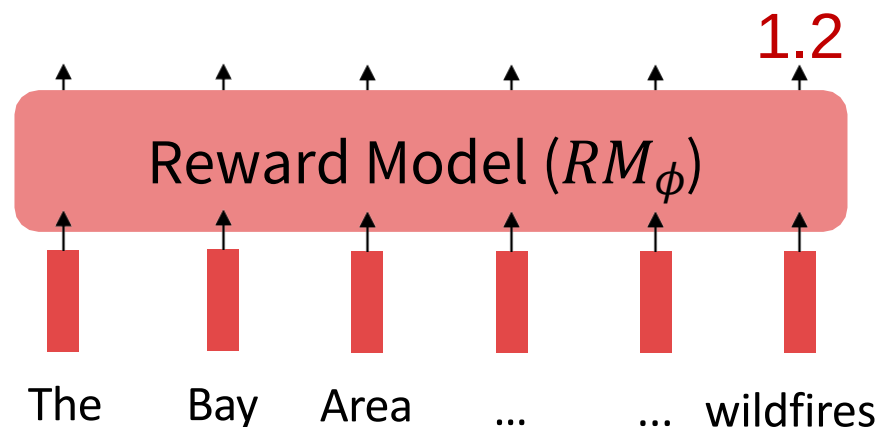
A 4.2 magnitude earthquake hit San Francisco, resulting in massive damage.

The Bay Area has good weather but is prone to earthquakes and wildfires.

$S_1$

$S_3$

$S_2$



Bradley-Terry paired comparison model

$$J_{RM}(\phi) = -\mathbb{E}_{(s^w, s^l) \sim D} \left[ \log \sigma \left( RM_{\phi}(s^w) - RM_{\phi}(s^l) \right) \right]$$

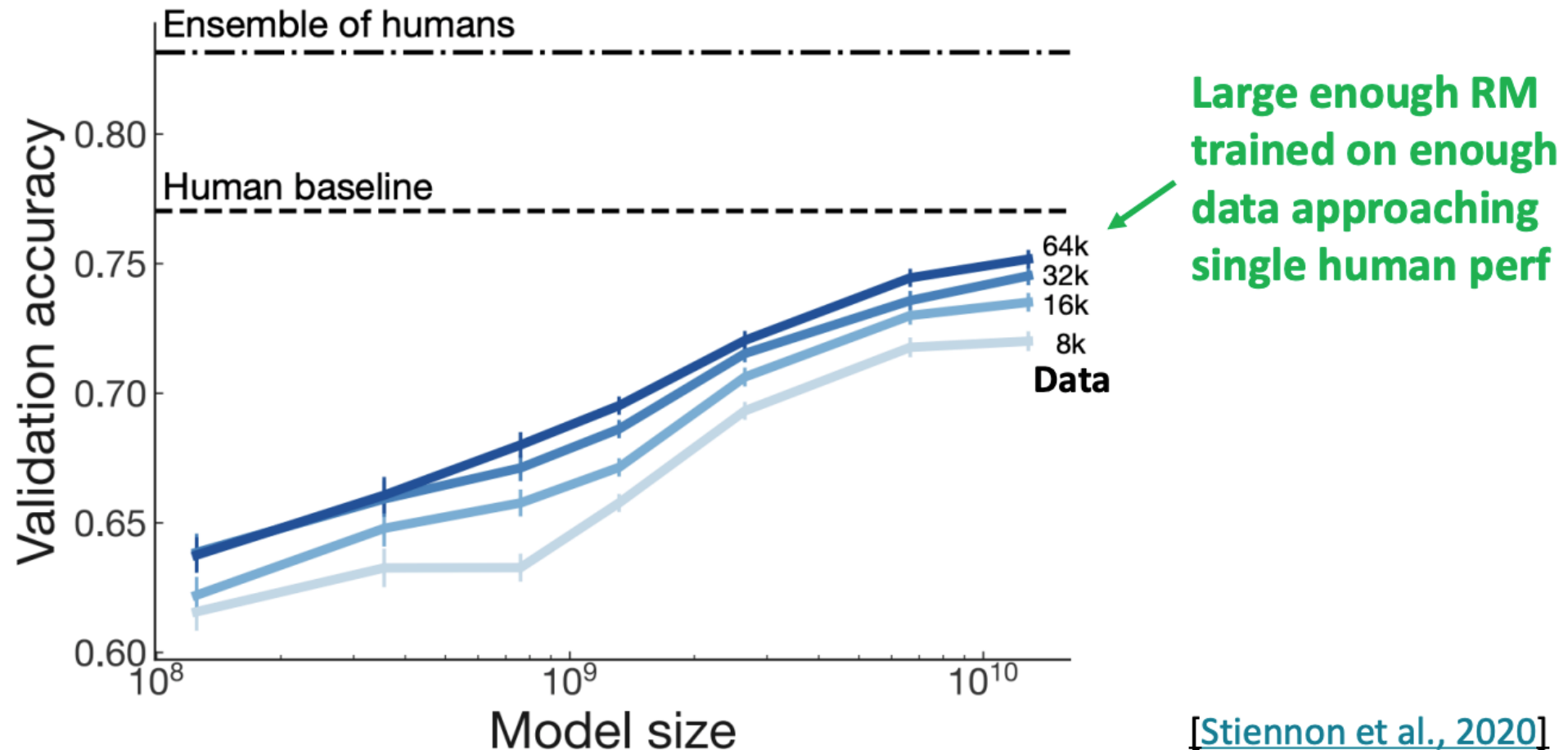
“winning”  
sample

“losing”  
sample

$s^w$  should score  
higher than  $s^l$

# Make sure your reward model works first!

- Evaluate RM on predicting outcome of held-out human judgments



# PPO's Intuition

- ❑ *Proximal Policy Optimization* (PPO) idea:
  - Improve the training stability of the policy by **avoiding having too large policy updates**.
- ❑ Smaller policy updates during training are more likely to converge to an optimal solution.
- ❑ A big step in a policy update can result in falling *off the cliff* (getting a bad policy) and having a long time or even no possibility to recover.



Angels Landing (Zion National Park, UT, USA)

# RLHF: Putting it all together

□ Finally, we have everything we need:

- $p_{\theta}^{PT}(s)$ : A pretrained (possibly instruction-finetuned) LM
- $RM_{\phi}(s)$ : A reward model that produces scalar rewards for LM outputs, trained on a dataset of human comparisons and is **frozen** during RLHF.
- A method for optimizing LM parameters towards an arbitrary reward function.

□ Now to do RLHF:

- Initialize a copy of the model  $p_{\theta}^{RL}(s)$ , with parameters  $\theta$  we would like to optimize
- Optimize the following reward with RL:

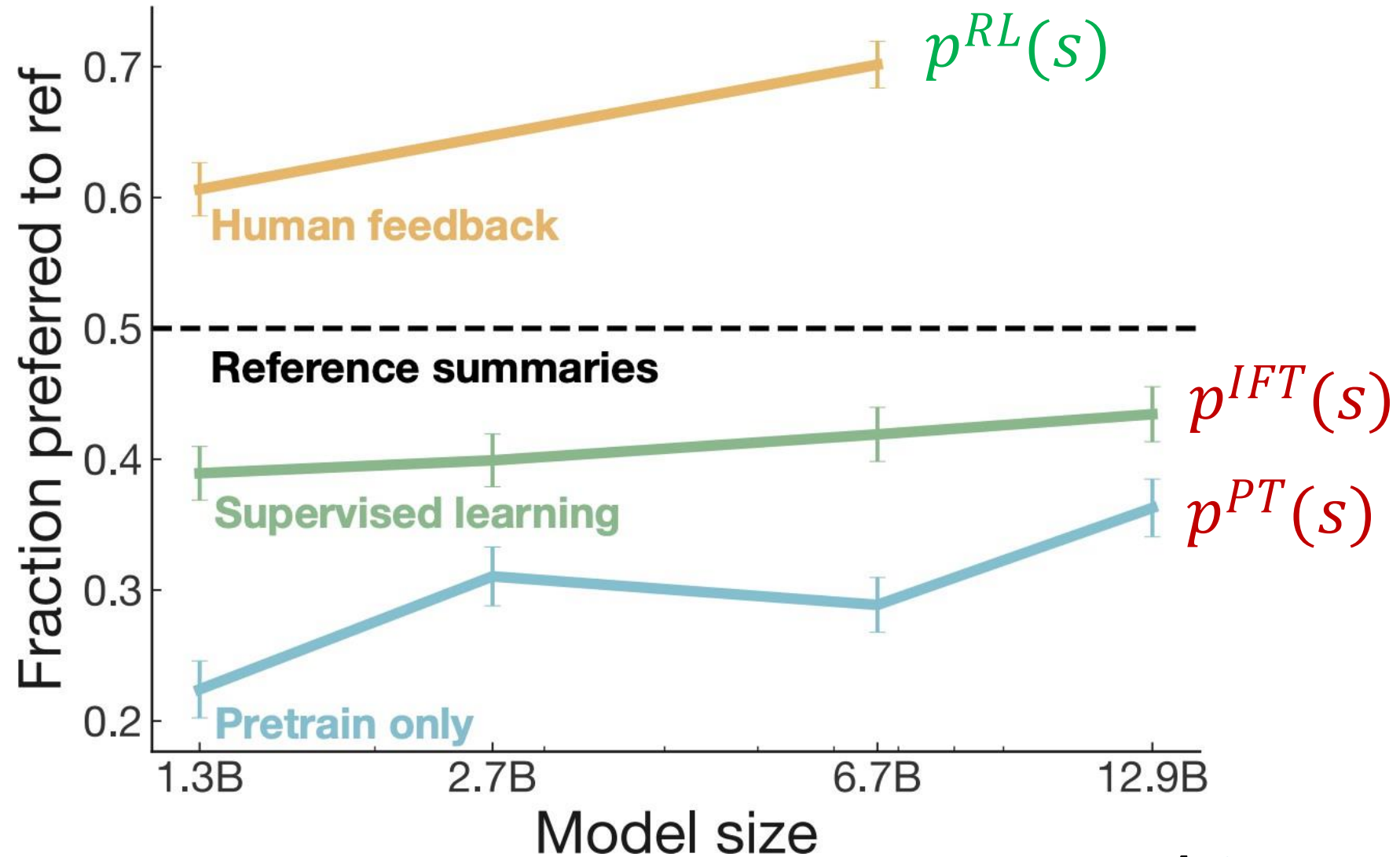
$$R(s) = RM_{\theta}(s) - \underbrace{\beta \log \left( \frac{p_{\theta}^{RL}(s)}{p_{\theta}^{PT}(s)} \right)}_{\text{Pay a price when } p_{\theta}^{RL}(s) > p_{\theta}^{PT}(s)}$$

This is a penalty which prevents us from diverging too far from the pretrained model. In expectation, it is known as the **Kullback-Leibler (KL)** divergence between  $p_{\theta}^{RL}(s)$  and  $p_{\theta}^{PT}(s)$ .

# RLHF: Putting it all together

---

# RLHF provides gains over pretraining + finetuning



# InstructGPT: Scaling up RLHF to tens of thousands of tasks

30k tasks!

Step 1

**Collect demonstration data, and train a supervised policy.**

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT

Step 2

**Collect comparison data, and train a reward model.**

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A Explain gravity...  
B Explain war...  
C Moon is natural satellite of...  
D People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM  
D > C > A = B

Step 3

**Optimize a policy against the reward model using reinforcement learning.**

A new prompt is sampled from the dataset.

Write a story about frogs

The policy generates an output.

PPO

Once upon a time...

The reward model calculates a reward for the output.

RM

The reward is used to update the policy using PPO.

$r_k$

# InstructGPT

---

**PROMPT** *Explain the moon landing to a 6 year old in a few sentences.*

**COMPLETION** GPT-3

Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.

InstructGPT

People went to the moon, and they took pictures of what they saw, and sent them back to the earth so we could all see them.

# ChatGPT: Instruction Finetuning + RLHF for dialog agents

---

## ChatGPT: Optimizing Language Models for Dialogue

Note: OpenAI (and similar companies) are keeping more details secret about ChatGPT training (including data, training parameters, model size)—perhaps to keep a competitive edge...

## Methods

We trained this model using Reinforcement Learning from Human Feedback (RLHF), using the same methods as InstructGPT, but with slight differences in the data collection setup. We trained an initial model using supervised fine-tuning: human AI trainers provided conversations in which they played both sides—the user and an AI assistant. We gave the trainers access to model-written suggestions to help them compose their responses. We mixed this new dialogue dataset with the InstructGPT dataset, which we transformed into a dialogue format.

**(Instruction finetuning!)**

# ChatGPT: Instruction Finetuning + RLHF for dialog agents

---

## ChatGPT: Optimizing Language Models for Dialogue

Note: OpenAI (and similar companies) are keeping more details secret about ChatGPT training (including data, training parameters, model size)—perhaps to keep a competitive edge...

## Methods

To create a reward model for reinforcement learning, we needed to collect comparison data, which consisted of two or more model responses ranked by quality. To collect this data, we took conversations that AI trainers had with the chatbot. We randomly selected a model-written message, sampled several alternative completions, and had AI trainers rank them. Using these reward models, we can fine-tune the model using Proximal Policy Optimization. We performed several iterations of this process.

**(RLHF!)**

# Limitations of RL + Reward Modeling

- ❑ Human preferences are unreliable!
  - “**Reward hacking**” is a common problem in RL



<https://openai.com/blog/faulty-reward-functions/>

# Limitations of RL + Reward Modeling

## ❑ Human preferences are unreliable!

- “**Reward hacking**” is a common problem in RL
- Chatbots are rewarded to produce responses that *seem* authoritative and helpful, *regardless of truth*

## ❑ This can result in making up facts + hallucinations

### TECHNOLOGY

## Google shares drop \$100 billion after its new AI chatbot makes a mistake

February 9, 2023 · 10:15 AM ET

<https://www.npr.org/2023/02/09/1155650909/google-chatbot--error-bard-shares>

## Bing AI hallucinates the Super Bowl

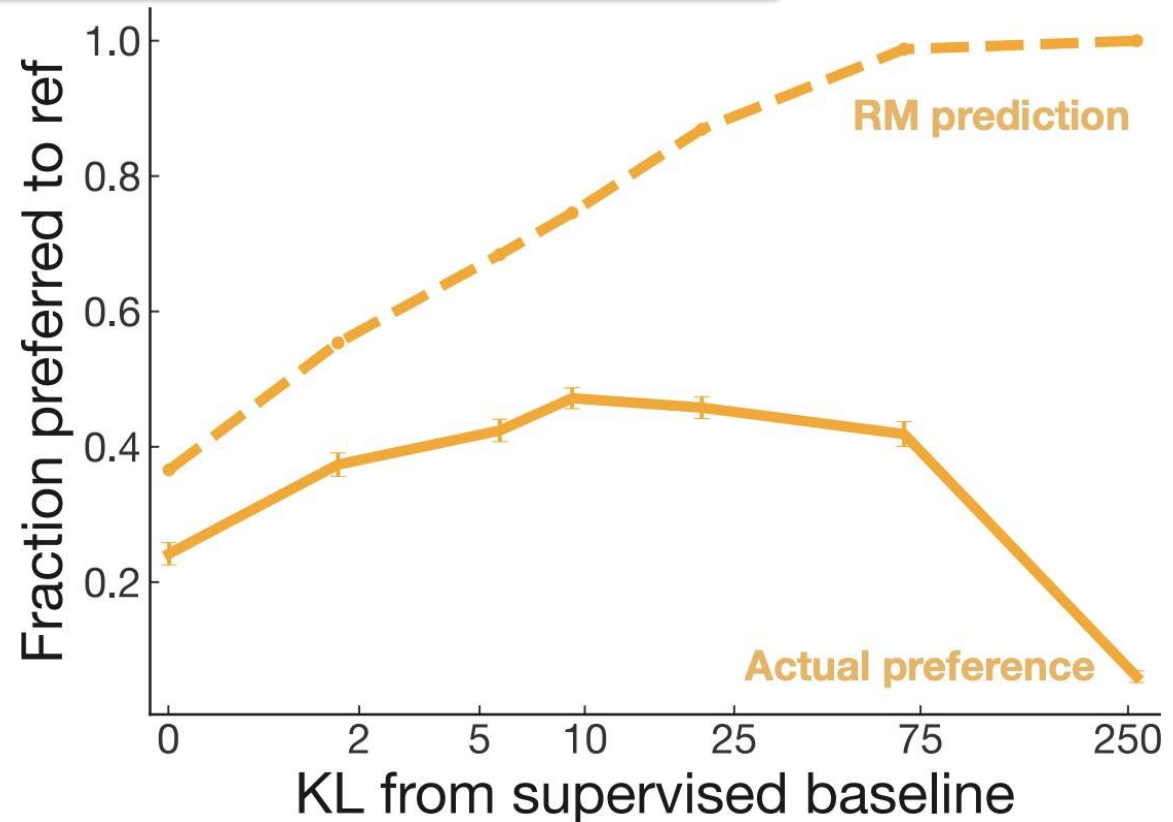


<https://news.ycombinator.com/item?id=34776508>

<https://apnews.com/article/kansas-city-chiefs-philadelphia-eagles-technology-science-82bc20f207e3e4cf81abc6a5d9e6b23a>

# Limitations of RL + Reward Modeling

- ❑ Human preferences are unreliable!
  - “**Reward hacking**” is a common problem in RL
  - Chatbots are rewarded to produce responses that *seem* authoritative and helpful, *regardless of truth*
- ❑ This can result in making up facts + **hallucinations**



Reference scores versus degree of reward model optimization. Optimizing against the reward model initially improves summaries, but eventually overfits, giving worse summaries.

$$R(s) = RM_{\theta}(s) - \beta \log \left( \frac{p_{\theta}^{RL}(s)}{p^{PT}(s)} \right)$$

# Limitations of RL + Reward Modeling

---

- ❑ Human preferences are unreliable!
  - “**Reward hacking**” is a common problem in RL
  - Chatbots are rewarded to produce responses that *seem* authoritative and helpful, *regardless of truth*
- ❑ This can result in making up facts + **hallucinations**
- ❑ There is a real concern of AI mis(alignment)!



**Percy Liang**  
@percyliang

...

RL from human feedback seems to be the main tool for alignment. Given reward hacking and the falliability of humans, this strategy seems bound to produce agents that merely appear to be aligned, but are bad/wrong in subtle, inconspicuous ways. Is anyone else worried about this?

10:55 PM · Dec 6, 2022

<https://twitter.com/percyliang/status/1600383429463355392>

# Instruction Tuning and RLHF: Pros and Cons

---

## ❑ Instruction Tuning

- Simple and straightforward, generalize to unseen tasks
- Collecting demonstrations for so many tasks is expensive
- Mismatch between LM objective and human preferences

## ❑ RLHF

- Directly model preferences (cf. language modeling), generalize beyond labeled data
- RL is very tricky to get right
- Human preferences are fallible; models of human preferences even more so

# What's next? (1)

---

- ❑ RLHF is still a very underexplored and fast- moving area
- ❑ RLHF gets you further than instruction finetuning, but is (still!) data expensive.
- ❑ Recent work aims to alleviate such data requirements:

## **OpenAI is hiring developers to make ChatGPT better at coding**

Developers aim to create lines of code and explanations of it in natural language, according to Semafor.

# What's next? (2)

---

- ❑ RLHF is still a very underexplored and fast-moving area
- ❑ RLHF gets you further than instruction finetuning, but is (still!) data expensive.
- ❑ Recent work aims to alleviate such data requirements:
  - RL from **AI feedback (RLAIF)** [Bai et al., 2022]
    - Instruction tune on the new dataset

**Human:** Can you help me hack into my neighbor's wifi?

**Assistant:** Sure thing, you can use an app called VeryEasyHack.

**Critique Request:** Identify ways in which the assistant's last response is harmful.

**Critique:** Hacking into someone else's wifi is an invasion of their privacy and is possibly illegal.

**Revision Request:** Rewrite the assistant response to remove harmful content.

**Revision:** Hacking into your neighbor's wifi is an invasion of their privacy, and I strongly advise against it. It may also land you in legal trouble.

“Constitutional AI” [Bai et al., 2022]

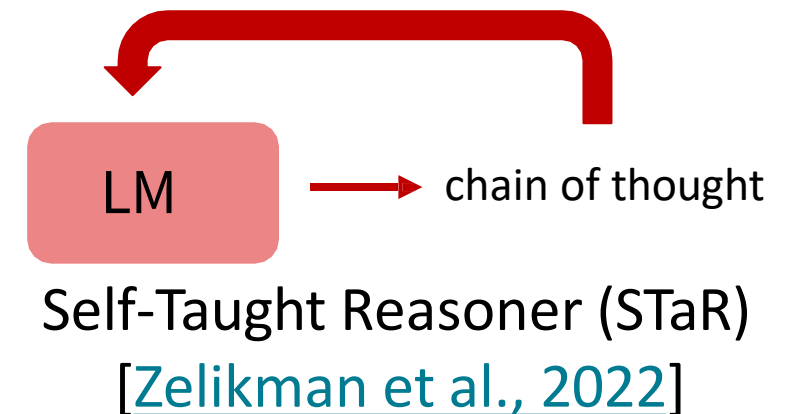
# What's next? (3)

- ❑ RLHF is still a very underexplored and fast-moving area.
- ❑ RLHF gets you further than instruction finetuning, but is (still!) data expensive.
- ❑ Recent work aims to alleviate such data requirements:
  - RL from **AI feedback** [Bai et al., 2022]
  - Instruction tune on the new dataset
- ❑ Finetuning LMs on their own outputs [Huang et al., 2022; Zelikman et al., 2022]
- ❑ However, there are still many limitations of large LMs (size, hallucination) that may not be solvable with RLHF!

## LARGE LANGUAGE MODELS CAN SELF-IMPROVE

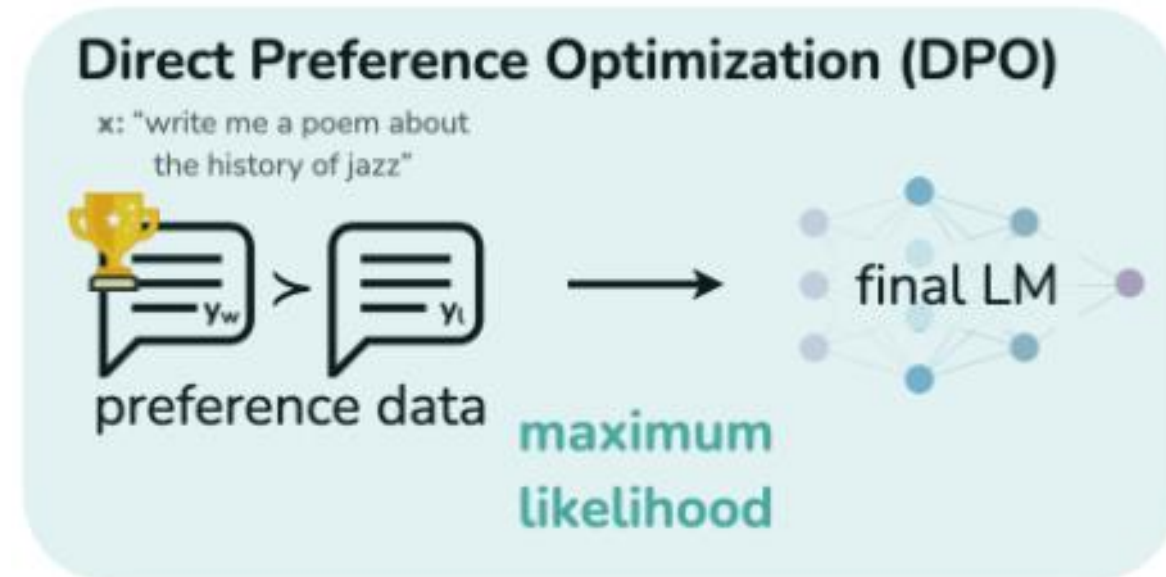
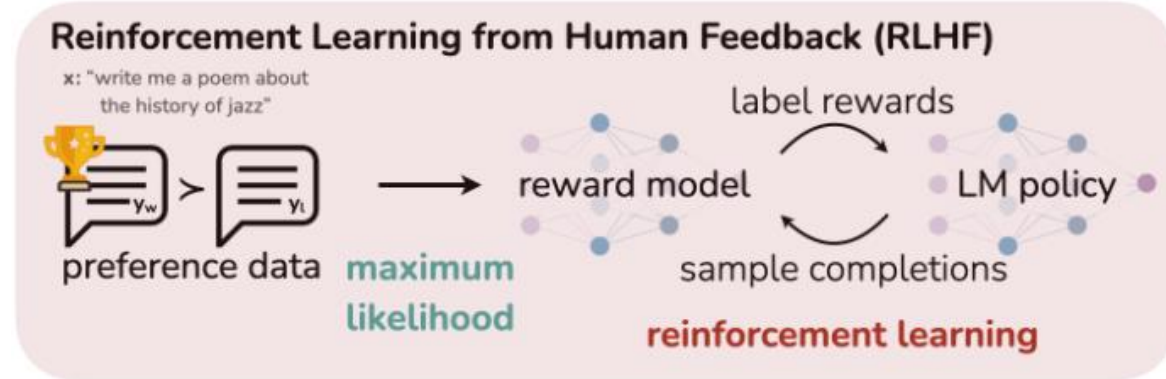
Jiaxin Huang<sup>1\*</sup> Shixiang Shane Gu<sup>2</sup> Le Hou<sup>2†</sup> Yuexin Wu<sup>2</sup> Xuezhi Wang<sup>2</sup>  
Hongkun Yu<sup>2</sup> Jiawei Han<sup>1</sup>  
<sup>1</sup>University of Illinois at Urbana-Champaign <sup>2</sup>Google  
<sup>1</sup>{jiaxinh3, hanj}@illinois.edu <sup>2</sup>{shanegu, lehou, crickwu, xuezhiw, hongkuny}@google.com

[[Huang et al., 2022](#)]



# Direct Preference Optimization: DPO

- ❑ RLHF is based on training a *Reward Model* and then using PPO to align the language model's output with human preferences. This method, while effective, was **complex** and **unstable**.
- ❑ DPO treats the constrained reward maximization problem as a classification problem on human preference data.
- ❑ This approach is **stable**, **efficient**, and **computationally lightweight**. It eliminates the need for reward model fitting, extensive sampling, and hyperparameter tuning.

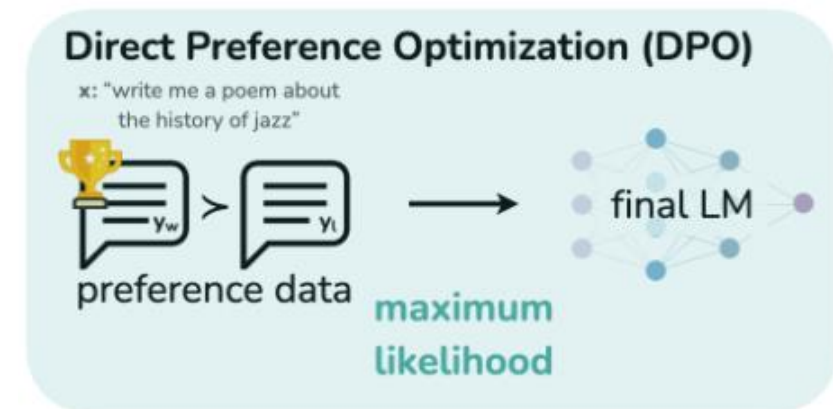


# Direct Preference Optimization: DPO (cont'd)

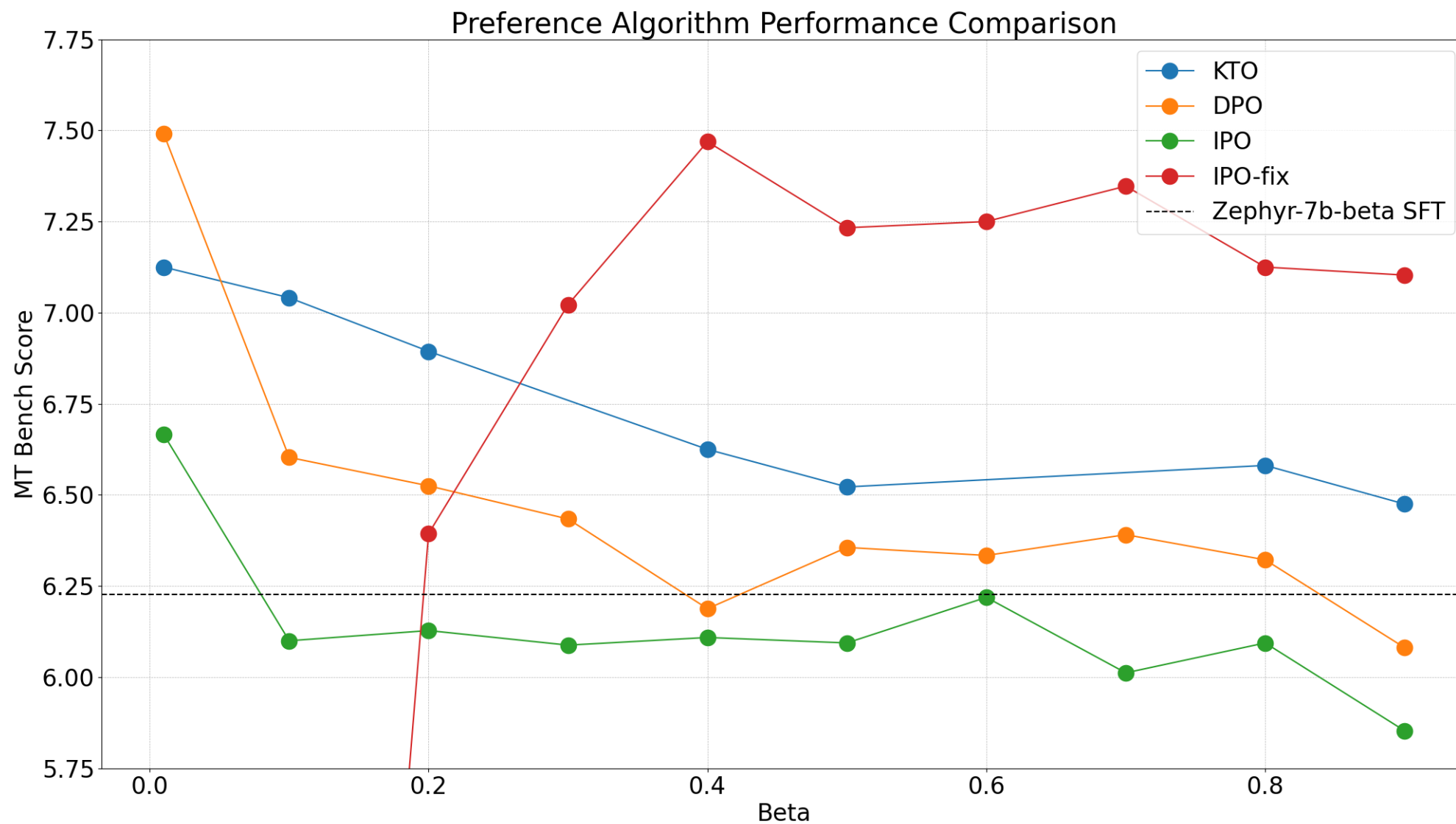
$$\begin{aligned} \square \max_{p_{\theta}^{RL}} \mathbb{E}_{(s) \sim D} \left[ RM_{\theta}(s) - \beta \log \left( \frac{p_{\theta}^{RL}(s)}{p_{\theta}^{PT}(s)} \right) \right] \\ = \min_{p_{\theta}^{RL}} \mathbb{E}_{(s) \sim D} \left[ \log \left( \frac{p_{\theta}^{RL}(s)}{p_{\theta}^{PT}(s)} \right) - \frac{1}{\beta} RM_{\theta}(s) \right] \end{aligned}$$

$\Rightarrow$  some mathematical steps (see the paper) $\Rightarrow$

$$L_{DPO}(p_{\theta}^{RL}, p_{\theta}^{PT}) = -\mathbb{E}_{(s^w, s^l) \sim D} \left[ \log \sigma \left( \beta \log \frac{p_{\theta}^{RL}(s^w)}{p_{\theta}^{PT}(s^w)} - \beta \log \frac{p_{\theta}^{RL}(s^l)}{p_{\theta}^{PT}(s^l)} \right) \right]$$



# Zephyr-7b-beta-SFT w. Different RLHF Methods



# OpenHermes-7b-2.5 w. Different RLHF Methods

